AIST4010 Lecture 14: RNN++ — Review Notes

1 Back-propagation through time

Prior knowledge for the hidden state model: $h_t = f(x_t, h_{t-1})$ computes the state, and $o_t = g(h_t, w_o)$ the output. Classification uses the last hidden state; regression may predict at every position; any model conditioned on previous values is autoregressive.
Training with gradient descent: initialize all the weights to random values, run the forward pass over all the training data to compute Y, then update

$$w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \alpha \frac{\partial \operatorname{Div}(Y, d)}{\partial w_{ij}^{(k)}}$$

and repeat until no more update. RNNs are trained with back-propagation and the common optimizers, but the recurrent dependency makes the gradient computation tricky.
Chain rule refresher: for a nested function $y = f(g(x))$, $dy/dx = (df/dg(x))(dg(x)/dx)$; for a distributed nesting $y = f(g_1(x), \ldots, g_M(x))$, the total derivative is the sum of the contributions through each path, $dy/dx = \sum_j (dy/dz_j)(dz_j/dx)$.

1.1 Gradient flow through the recurrent cell

With $h_t = f(x_t, h_{t-1}, w_h)$, $o_t = g(h_t, w_o)$ and the mean loss $L = \frac{1}{T}\sum_{t=1}^{T} l(y_t, o_t)$ over all T steps, the gradient with respect to w_h flows through every step:

$$\frac{\partial L}{\partial w_h} = \sum_{t=1}^{T} \frac{1}{T} \cdot \frac{\partial l(y_t, o_t)}{\partial o_t} \cdot \frac{\partial g(h_t, w_o)}{\partial h_t} \cdot \frac{\partial h_t}{\partial w_h}, \qquad \frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial w_h}.$$

Expanding the recursion gives a product of the Jacobians $\partial f/\partial h_{j-1}$ along the chain. For the linear cell $h_t = W_{hx}x_t + W_{hh}h_{t-1}$, $o_t = W_{qh}h_t$, the terms are powers of W_{hh} raised along the chain:

$$\frac{\partial L}{\partial h_t} = \sum_{i=t}^{T} W_{hh}^{T-i} W_{qh}^{T} \cdot \frac{\partial L}{\partial o_{T+t-i}}.$$

For a large spectral radius the product explodes, for a small one it vanishes. Long chains are slow, may explode or vanish — this motivates truncation.

1.2 Truncation

Full back-propagation through all T steps is slow and numerically fragile; gradients can explode or vanish.

- Truncating time steps: stop the sum after τ steps so the longest dependency is at most τ.
- Randomized truncation: stop after a random number of steps, which biases the estimate while stabilizing it.

2 Gated recurrent units (GRU)

Prior knowledge: text can be long; early observations may be important (a naive cell may decay them away); some information is meaningless and should be skipped; a logic break should reset the state.
The plain recurrence

$$H_t = \phi\big(X_t W_{xh} + H_{t-1} W_{hh} + b_h\big)$$

always mixes everything; we need gates that turn information on or off to memorize, skip, and reset.

2.1 Building the gates

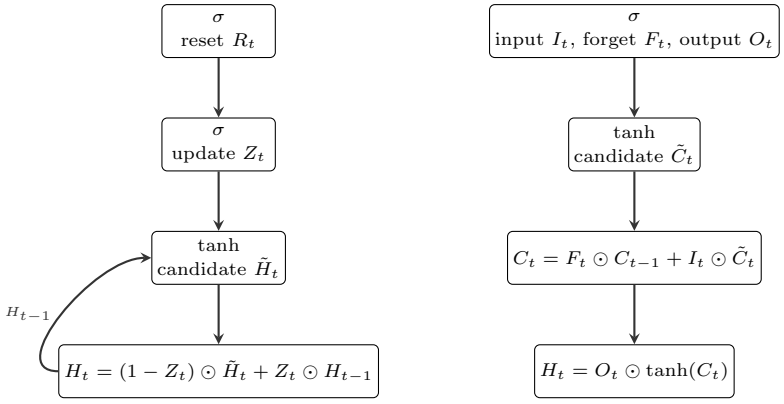
- *First attempt (reset only):* $H_t = \phi\big(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h\big)$ with a binary vector R_t that zeroes out some previous states. It can forget, but cannot skip the current input.
- *Second attempt (reset + input gate):* $H_t = \phi\big((Z_t \odot X_t) W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h\big)$. If both gates close, the cell collapses to $\phi(b_h)$ — information is lost.
- *Third attempt:* add a pass-through term $P_t \odot H_{t-1}$. Memorizing must be exclusive from resetting, and the pass-through should be complementary to skipping.
- *Fourth attempt (the GRU update):* replace the separate controls with one interpolation between a candidate state and the previous state.

2.2 GRU equations

Reset gate R_t decides how much of H_{t-1} enters the candidate; update gate Z_t interpolates between the candidate and H_{t-1}. Gate outputs are near 0 or 1 from the sigmoid σ, while the candidate uses tanh:

$$R_t = \sigma\big(X_t W_{xr} + H_{t-1} W_{hr} + b_r\big), \qquad Z_t = \sigma\big(X_t W_{xz} + H_{t-1} W_{hz} + b_z\big),$$
$$\tilde{H}_t = \tanh\big(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h\big), \qquad H_t = (1 - Z_t) \odot \tilde{H}_t + Z_t \odot H_{t-1}.$$

Reset gates help capture short-term dependencies in sequences; update gates help capture long-term dependencies in sequences.



(a) GRU: two gates, one interpolation. (b) LSTM: a dedicated memory cell.

Figure 1: GRU interpolates between a candidate state and the previous state; LSTM keeps a dedicated cell state that is updated additively through forget and input gates.

3 Long short-term memory (LSTM)

Memory is important: could we have a dedicated memory cell? The answer combines three gates — input, forget, and output — with a cell state that flows through time.
The cell state C_t is updated additively, so gradients can flow without repeated multiplication; this alleviates vanishing and exploding gradients:

$$I_t = \sigma\big(X_t W_{xi} + H_{t-1} W_{hi} + b_i\big), \quad F_t = \sigma\big(X_t W_{xf} + H_{t-1} W_{hf} + b_f\big), \quad O_t = \sigma\big(X_t W_{xo} + H_{t-1} W_{ho} + b_o\big),$$
$$\tilde{C}_t = \tanh\big(X_t W_{xc} + H_{t-1} W_{hc} + b_c\big), \quad C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t, \quad H_t = O_t \odot \tanh(C_t).$$

The memory cell C_t is very similar to the candidate hidden state in GRU, but now it is protected by its own forget and input gates.

4 Deep, bidirectional, and sequence-to-sequence models

4.1 Deep RNN/LSTM/GRU

A single recurrent layer may be too simple. Stacking layers increases capacity: lower layers learn shorter-term dynamics, higher layers learn high-level trends. The l-th hidden layer follows

$$H_t^{(l)} = \phi_l\big(H_t^{(l-1)} W_{xh}^{(l)} + H_{t-1}^{(l)} W_{hh}^{(l)} + b_h^{(l)}\big), \qquad O_t = \sigma\big(H_t^{(L)} W_{ho} + b_o\big).$$

4.2 Bidirectional model

Downstream information can be helpful: "I am ___ hungry, and I can eat half a pig" versus "I am ___ hungry, because I have already eaten half a pig." A bidirectional model runs two recursions, forward over $H_{t-1}^{(f)}$ and backward over $H_{t+1}^{(b)}$, and concatenates both for the output

$$O_t = \sigma\big(\mathrm{cat}(H_t^{(f)}, H_t^{(b)})\, W_{ho} + b_o\big).$$

Limitations: it cannot do forward prediction (training sees both directions, testing only one, giving poor performance). It fits filling in missing words, annotating tokens such as named-entity recognition, and sequence encoding for machine translation. It is very, very slow: the forward pass requires both a forward and a backward recursion.

4.3 Sequence-to-sequence model

Machine translation maps a sequence from one language to another, e.g. "Thank you" $\to$ "xie xie ni", usually with word-level tokenization. Sequences have different lengths, so a direct position-wise mapping does not exist. An encoder transforms the input into a state of fixed shape; a decoder maps that state to a variable-length sequence.

Encoder: $h_t = f(x_t, h_{t-1})$ and the context $c = q(h_1, \ldots, h_T)$, for example $c = h_T$. Decoder: $s_{t'} = g(y_{t'}, c, s_{t'-1})$. The loss function is cross-entropy for each output position (a classification), and prediction stops when "`<eos>`" is generated.

Evaluation: BLEU (Bilingual Evaluation Understudy). p_n is the precision of n-grams, the number of matched n-grams divided by the number of n-grams in the predicted sequence. Example: target ABCDEF, predicted ABBCD, so $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, $p_4 = 0$. BLEU multiplies the n-gram precisions with a brevity penalty; the higher, the better:

$$\mathrm{BLEU} = \exp\Big(\min\big(0,\, 1 - \tfrac{\mathrm{len}_{\mathrm{target}}}{\mathrm{len}_{\mathrm{pred}}}\big)\Big) \prod_{n=1}^{k} p_n^{1/2^n}.$$

4.4 Search strategies

- Greedy search picks the most probable token at each step, but a locally best choice can miss a globally better path: $0.5 \cdot 0.4 \cdot 0.4 \cdot 0.6 = 0.048$ versus $0.5 \cdot 0.3 \cdot 0.6 \cdot 0.6 = 0.054$.
- Exhaustive search over $|D|$ tokens and L steps costs $|D|^L$ — infeasible for e.g. $|D| = 10000$, $L = 10$.
- Beam search: keep k partial sequences, expand each by the top candidates, and select the k best among the $k|D|$ possibilities at every step.

5 Attention

In seq2seq the hidden state is very important, yet there is only one hidden state aggregating the sentence; it cannot guarantee a proper representation for each word. The natural idea: encode each word by considering its context.

Representation with neighbor information: an average $f(x) = \frac{1}{n}\sum_i y_i$, or a weighted average $f(x) = \sum_i \alpha(x, x_i)\, y_i$ where α reflects similarity or distance. In the most general form this is attention.

5.1 Attention for machine translation

Hidden state of each word: consider interactions with all other words directly. A query hidden state gathers the states of the tokens through a weighted average,

$$f(x) = \sum_i \alpha(x, x_i)\, y_i.$$

Three questions: how to encode characters (keys)? how to define the values? how to define the attention weight function?
Keys: one-hot encoding, or a fully connected network applied to the one-hot vector. Values: a fully connected network over the one-hot vector.
Attention weights (dot-product):

$$\alpha(q, k_i) = \frac{\exp\big(a(q, k_i)\big)}{\sum_j \exp\big(a(q, k_j)\big)}, \qquad a(q, k) = \frac{q^\top k}{\sqrt{d}}.$$

Loss function: cross-entropy for each word prediction. Training: the common optimizers with back-propagation.

5.2 Self-attention

For each input x_i in the sentence,

$$y_i = \sum_j \alpha(x_i, x_j)\, v_j.$$

Each output y_i is a hidden state for downstream tasks, attending against the sentence itself — including the token itself. It can be computed in parallel for all tokens; encoding and embedding are the most important things for NLP.

5.3 Position encoding

Attention pools over the sequence without any order information. Time series, sequences, and text all carry order, so position must be injected. For $X \in \mathbb{R}^{n\times d}$, a position matrix $P \in \mathbb{R}^{n\times d}$ adds sine and cosine features of the row index, with frequencies that decay along the dimension:

$$p_{i,2j} = \sin\Big(\frac{i}{10000^{2j/d}}\Big), \qquad p_{i,2j+1} = \cos\Big(\frac{i}{10000^{2j/d}}\Big).$$

It looks like a weird function but people use it; it encodes both absolute and relative positional information. The input dimension equals the raw encoding dimension plus the position encoding dimension.

5.4 Multi-head attention and the Transformer

- Input of an attention layer: $L\times$ (position embedding + one-hot); output: $L\times$ hidden-state dimension.
- To increase complexity: multi-head attention and stacking more attention layers.
- Multi-head attention projects queries, keys, and values into different representation subspaces, attends in each head, and concatenates the outputs (dimension $\times n_{\text{head}}$).
- A Transformer block: multi-head attention learns the hidden state; fully connected layers control the dimension (for easy implementation); masked attention ignores padding positions; multiple blocks increase complexity.